

Chapter 5

1. It is easy to reason about what the fork tree will look like. Each process that forks gets a branch and a process that exit is removed from the tree.
2. I noticed that when the fork chance is low but actions is high the depth of the tree remains pretty shallow, when the fork chance is much higher then the tree grows much larger since more forks means more branches on the tree. With the fork chance high I was unable to keep track of the tree in my head without checking.
3. With the -t flag I am definitely able to tell which action was taken by comparing the tree before and after.
4. After a process with one or more child processes I initially thought that you would not be able to exit. This was not the case, I noticed the children as child processes of the parent. This seems like reasonable default behavior since they have no direct parent anymore. When I used the -R flat I noticed something very strange. This command said it would “Reparent” and it did exactly that. Instead of putting the child process back with the parent it was just shifted one parent up. So when c was removed it’s e became a child of it’s parent b instead of a. It would seem that the -R flag reparents to the closest surviving ancestor instead of the root process (a).
5. I noticed I was seemingly able to always reconstruct the final tree. I believe this is because tracing step by step actions forward is always deterministic. Even when using many different seeds this still held.
6. With -F and -t I wasn’t always able to trace back from the tree. In noticed that whenever an exit() had happened somewhere information was lost. It is difficult to tell exactly what happened. In situations where multiple siblings were created independently it was also difficult to tell.

Chapter 5 Coding

See the c files in my `coding_1` directory

1. `fork_and_var.c`. The variable should have the same value. This is because the child receives an exact copy of the parent address space when `fork()` gets called. When the value gets changed nothing happens in the other address space because the two are isolated from one another by the OS.
2. `fork_and_file.c`. Yes, the child process has a copy of the parent’s file descriptor. When they write they do not overwrite each other’s data because of the file offset. The order that the data appears would seem to be not easily predicted, this is up to how the OS decides to schedule.
3. `fork_child_first.c`. You can use some form of synchronization for this to work. Pipes are an example of this.

4. `fork_and_exec.c`. There are so many variants to provide flexibility for the programmer based on what data they may have access through and how they might want to set things up. The letters tell us how different args and environments are passed.
5. `fork_and_wait_returns.c`. `Wait` returns the pid of the child process that is terminated. If you use `wait` in the child it fails and returns -1 because this isn't allowed.
6. `fork_and_waitpid.c`. This is very useful when a parent generates multiple children without waiting on all of them. The parent can wait for one specific child to finish rather than all.
7. `close_stdout.c`. The text doesn't appear in the terminal when the child closes the stdout before printing. `printf` can't write to the file descriptor, so a negative int is returned and the program continues.
8. `children_and_pipe.c`. The program I wrote is able to connect to the standard output of one and the standard input of the other using a pipe syscall.

Chapter 6 coding

1. I satisfied this assignment with 2 programs, `syscall_cost.c` and `context_switch_cost.c`. Here were the results:

```
ubuntu@cs5600:~/coding_1$ gcc syscall_cost.c && ./a.out
Syscall average: 1.107702 usec
```

```
ubuntu@cs5600:~/coding_1$ gcc context_switch_cost.c && ./a.out
Context switch average: 6.599585 usec
```

These results show me that syscalls are significantly more expensive than context switches. Mode switches (what syscalls are) are a few ops but address space doesn't need to be touched. Context switches are much more expensive because there is a lot of state saving, scheduling, restoring, ect. that has to happen.

Chapter 7

1. When I ran the first sim with FIFO the average response was 200 and the average turnaround was 400. With SJF it was the same.
2. With FIFO: avg response: 133.33 avg turnaround: 333.33. With SJF avg response: 133.33, avg turnaround 333.33.
3. RR: avg response 1.00, Turnaround time 466.67
4. SJF is equal to FIFO when jobs are already shortest to longest (sorted). They are also equal in cases where all jobs are the same length.

5. they'll have the same response times only when the RR quantum is greater than or equal to the length of the longest job. RR in this case runs them sequentially like SJF does.
6. As job lengths scale the average response time scales linearly with them.
7. As quantum increases the average response time gets higher because jobs in the back of the line have to wait longer for the first job to finish. If we were to represent this in an equation it would be the following: for n jobs with a quantum of q , the worst-case response time is $(n - 1) * q$.